

Data Modeling and Management

Given a scenario, identify best practices for various types of custom metadata and custom settings, and how to implement required solutions.

Table of Contents

-  [Custom Settings](#)
-  [Use Cases & Examples](#)
-  [Custom Metadata Types](#)
-  [Use Cases & Examples](#)
-  [Custom Settings vs. Custom Metadata Types](#)



After studying this topic, you should be able to:

-  Explain the difference between the types of custom settings, and difference between custom settings vs. custom metadata.
-  Identify the benefits, considerations, and limitations of using custom settings and custom metadata.
-  Demonstrate how to set up and access custom metadata and custom settings in Apex and declarative solutions.



Introduction

Custom settings and custom metadata types are used to store values which do not change frequently but need to be **frequently referenced** in Salesforce. The data from custom settings and custom metadata types is pulled into the **application cache** which allows for efficient access of the data and, in most cases, **does not count toward SOQL limit**. Both approaches support many different data types. Referencing custom settings and custom metadata types in business logic makes customizations easier to maintain and read, and reduces the number of places where changes need to be made. **Custom metadata types** are a newer concept and **are preferred over custom settings** in many cases. However, there are scenarios in which custom settings are still the preferred approach.



Custom Settings & Custom Metadata Types

Custom Settings

List and Hierarchy Custom Settings can be used to create custom sets of data and associate custom data for an organization, profile, or specific user.

Use Cases of Custom Settings

Custom Settings can be used when the data needs to be referenced frequently, but it does not change frequently or changes based on the logged-in user or their profile.

Using Custom Settings

Custom Settings can be created by navigating to 'Custom Settings' in Setup; the API name of a custom setting ends with '`_c`', similar to a custom object.



Custom Metadata Types

A Custom Metadata Type can be created to define the structure of application metadata; custom metadata records can be created, and relationships can be defined.

Use Cases of Custom Metadata Types

Custom Metadata Types can be used to store mappings between configuration data, business rules, and when configuration data is linked by relationships.

Using Custom Metadata Types

Custom Metadata Types can be created by navigating to 'Custom Metadata Types' in Setup; the API name of a custom metadata type ends with '`_mdt`'.

Custom Settings

Table of Contents



Custom Settings Overview

Custom settings are similar to custom objects in which **data** can be referenced in **declarative** and **programmatic** customizations.

❖ DATA

Custom settings allow **creation** of custom **sets of data** which can be exposed in application cache.

❖ USAGE

These settings can be used by **formula fields**, **validation rules**, **flows**, **Apex**, and the **SOAP API**.

❖ PACKAGES

Custom settings can be **included in packages**, but the **data will not be included**. To include data, Apex code will need to be used.



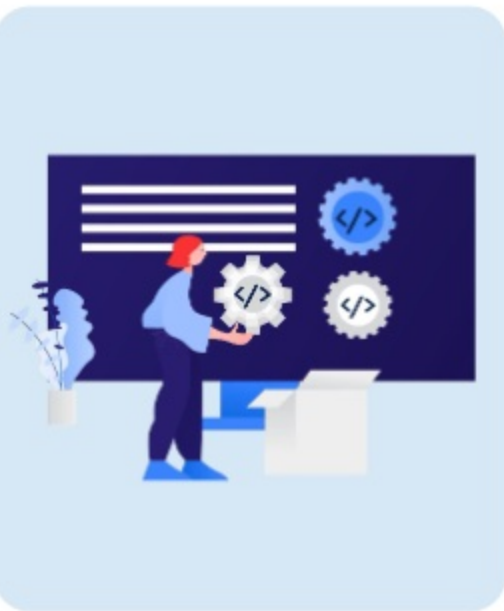
Custom Settings Overview

◆ VISIBILITY

To protect the value of Custom Settings in managed packages, **set Visibility to Protected**. However, changing the Visibility field is only allowed in **Developer** or **Scratch** orgs. This will prevent the target org from editing the values or accessing them via Apex or API.

◆ CACHED DATA

Custom settings can be used for quick and efficient access to cached custom data **without using SOQL queries** that count against the governor limits.



Custom Settings Overview

There are **two types** of custom settings:



LIST CUSTOM SETTINGS

Provides reusable static data that is the same for **all users** across the organization.



HIERARCHY CUSTOM SETTINGS

Uses built-in **hierarchical logic** to personalize settings based on specific **Profiles** or **Users**.

Custom Settings Overview



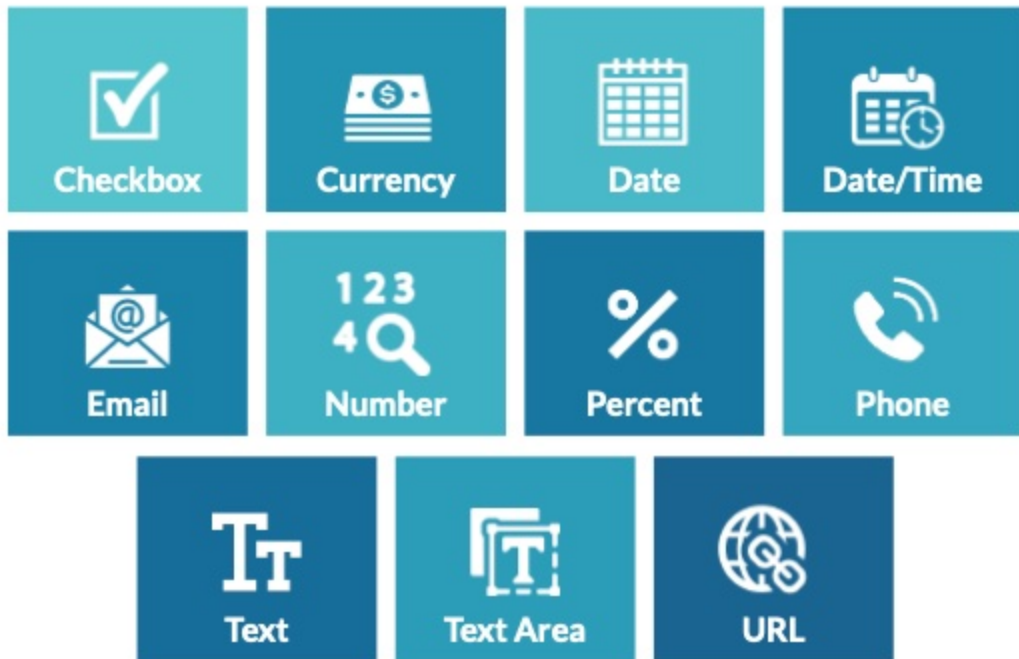
NOTE



For situations where **list custom setting** has been used in the past, **custom metadata types** are now the preferred approach.

Field Types Supported

The following are the field types **supported** by custom settings:



Using Custom Settings

In order to **use** custom settings, the **information** below is observed:



◆ SETUP MENU

Create custom settings by navigating to “**Custom Settings**” in Setup.

◆ MANAGE BUTTON

The *Manage* button can be used to **add**, **edit**, or **delete** values of a custom setting.

Using Custom Settings



◆ NAME FORMAT

Custom settings have the same naming format as custom objects. The **API name** ends in a “**_c**” suffix.

◆ EXAMPLE NAME

For example, a Custom Setting named “*Country Codes*” would be named **Country_Codes_c** by default.

Using Custom Settings



❖ FORMULA REFERENCE

To **access** custom settings, **prefix** the **API name** with **\$Setup** in validations or formulas (for example: `$Setup.Run_Validation_Rules__c`).

❖ APEX REFERENCE

To use in Apex, custom settings can be accessed the **same way** as a normal **Salesforce object** is accessed. Then **methods** specific to the type of custom setting (**list** or **hierarchy**) can be used.

Use Cases & Examples

Table of Contents



Use Cases of Custom Settings

The following are some **use cases** of Custom Settings:



FREQUENCY

Custom settings can be used for data that **does not change frequently** but is **referenced frequently**.



CURRENCY

A common **currency value** is needed.



COMMONALITY

A common **value** needs to be used in **formula** or **validation** fields.



USER SETTING

The value of the setting needs to **change** based on the **logged-in user** or the **user's profile**.



TRIGGER CONTROL

A custom setting can be used to **enable/disable** (programmatically) an Apex trigger event.



ORG SETTING

The specific value of the setting will **differ** for **every organization**.

Examples of Custom Settings

The following details some **example usages** of custom settings:

COUNTRY DATA

A **list** custom setting can be created to store a list of **countries** and their **ISO country codes** that can be used across the organization.



PLANS BY PROFILE

A **hierarchy** custom setting can be used to store **different incentives** and **bonus plans** for different user profiles for a compensation application.

Examples of Custom Settings

DOMAIN NAMES

A **list** custom setting can be created for storing **domain names** through which an organization receives **incoming emails**.



DISCOUNTS BY USER

A **hierarchy** custom setting can be created for **different discount percentages** that can be offered by different sales users in the organization.

Custom Settings Setup

A custom setting below called “*Run Validation Rules*” is used to **conditionally** run a **validation rule** depending on the **profile** of the current user.

Custom Setting

Run Validation Rules

Help for this Page ?

If the custom setting is a list, click **New** to add a new set of data. For example, if your application had a setting for country codes, each set might include the country's name and dialing code.

If the custom setting is a hierarchy, you can add data for the user, profile, or organization level. For example, you may want different values to display depending on whether a specific user is running the app, a specific profile, or just a general user.

EditDelete

▼ Default Organization Level Value

LocationFocus on ForceRun Validation ✓

Default for entire organization

View: Detailed View ▼ Edit | Create New View

Overrides for specific profiles/users

Action	Setup Owner ↑	Location	Run Validation	Name
View Edit Del	Standard Platform User	Profile	✓	Run Validation Rules (Profile)
View Edit Del	Martin Gessner	User	☐	Run Validation Rules (User)

Custom Settings Value

When defining a custom setting **value**, it can be **associated** with a profile or user. In this example, a "Run Validation" checkbox field is **enabled** for the selected **profile**.

Run Validation Rules Edit

[Help for this Page](#) 

Provide values for the fields you created. This data is cached with the application.

Edit Run Validation Rules

Save

Cancel

Run Validation Rules Information

 = Required Information

Location

 Profile ▾

Standard Platform User



Run Validation



Postal Code Validation Rule

The custom setting is then **referenced** below to **conditionally bypass** a validation rule for postal codes. The **API name** of the custom setting and the custom checkbox field is used.

Validation Rule Edit

SaveSave & NewCancel

Rule NameValid_Canadian_Postal_Code

Active☒

Description

Error Condition Formula

Example: Discount_Percent__c>0.30More Examples...

Display an error if Discount is more than 30%

If this formula expression is **true**, display the text defined in the Error Message area

Insert Field

Insert Operator

```
AND(  
  $Setup.Run_Validation_Rules__c.Run_Validation__c,  
  OR(BillingCountry = "CA", BillingCountry = "CAN", BillingCountry = "Canada"),  
  NOT (REGEX(  
    BillingPostalCode, "(?i) [ABCEGHJKLMNPRSTVXY] \\d[A-Z]? \\s? \\d[A-Z] \\d)" )  
)
```

Functions

-- All Function Categories --

ABS
ADDMONTHS
AND
BEGINS
BLANKVALUE
BR

Insert Selected Function

ABS(number)
Returns the absolute value of a

Close Date Validation Rule

In this example, a certain custom setting is used to **allow** administrators to **create** or **save** Opportunity records with close dates that are already in the **past**.

Validation Rule Edit Save Save & New Cancel

Rule Name

Active ☒

Description

Error Condition Formula

Example: [More Examples...](#)
Display an error if Discount is more than 30%
If this formula expression is **true**, display the text defined in the Error Message area

Insert Field Insert Operator ▼

```
AND(  
  $Setup.Admin_Overrides__c.Run_Validation_Rule__c,  
  CloseDate < TODAY()  
)
```

Check Syntax

Functions
-- All Function Categories -- ▼
ABS
ADDMONTHS
AND
BEGINS
BLANKVALUE
BR
Insert Selected Function
ABS(number)

Apex Trigger Validation

The code snippet below shows an Apex trigger that **references** a custom setting **value** to determine whether it should perform a **validation logic** when invoked.

```
trigger AccountTrigger on Account (before insert) {  
  
    // retrieve the user-specific information of this hierarchy custom setting  
    Run_Validation_Rules__c runVal = Run_Validation_Rules__c.getInstance();  
  
    If (runVal.Run_Validation__c) {  
        // perform validation steps only if the Run Validation checkbox is set  
    }  
}
```


Custom Metadata Types

[Table of Contents](#)



Custom Metadata Types Overview

Custom metadata types can be used to easily **customize**, **deploy**, or **update** application metadata.



PACKAGES

Custom metadata types allow the **structure** and **actual record data** to be included in a package.



CUSTOM FIELDS

Like custom settings, a custom metadata type consists of **custom fields** for storing attributes of **metadata**.



RELATIONSHIPS

Unlike custom settings, custom metadata types allow **creating relationships** between related types.

Custom Metadata Types Overview



FORMULA FIELDS

Records of a custom metadata type can be **referenced** in an advanced **formula field**.



MIGRATION

Custom metadata types can be **migrated** by using **change sets**.



VISIBILITY

The Visibility settings for custom metadata types can only be set in **Developer** or **Scratch** orgs.

Custom Metadata Types Overview

In managed packages, custom metadata types can be **protected** at the **Type**, **Record**, and **Field Level**.



PUBLIC TYPE

Public types can be accessed by **all Apex code** in an org.



PROTECTED TYPE

Protected types can **only** be accessed by the **package**.



PROTECTED RECORD

Protected records can only be accessed by code in the **same namespace** as the record or its type.



ADD VALUES

A combination of **Public type** and **Protected record** can be used to allow package subscribers to add their own values to a type, but **not** to modify the **type structure**.

Custom Metadata Types Overview

Fields have three levels of editing privileges:



PACKAGE DEVELOPER

A **package developer** can edit the field via package upgrade.



CUSTOMIZE APPLICATION

Any user with "Customize Application" **permission** can edit.
Package **upgrades will not override** the value in this case.

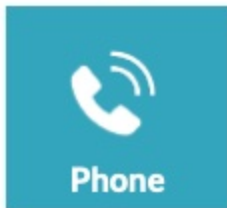
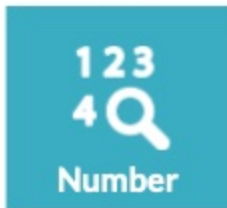
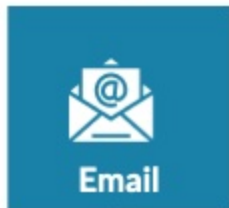
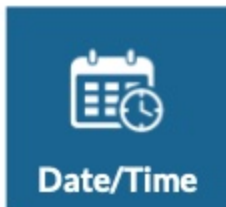
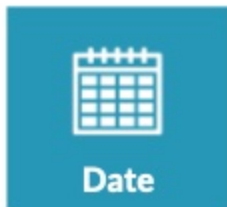


NONE

No one can edit, not even the package developer.

Field Types Supported

The following are the field types **supported** by custom metadata types:



Using Custom Metadata Types

In order to **use** custom metadata types, the **information** below is observed:



SETUP MENU

Custom metadata types can be created by navigating to '**Custom Metadata Types**' in Setup.



CONFIGURATION

The **data type** has to be selected, the **fields** are added, and then the **data** can be added.



POPULATION

Data can be **created manually** or loaded via the **custom metadata loader**.

Using Custom Metadata Types



NAME FORMAT

Custom metadata types append “**_mdt**” to the **API name** instead of “__c”.



EXAMPLE NAME

For example, a metadata type named “*Country Codes*” would be named **Country_Codes_mdt** by default.



DATA ACCESS

Custom metadata records can be accessed through **Apex**, **formulas**, or **flows**. To access the records in Setup, additional permissions are required.

Custom Metadata Type Methods

Custom metadata types provide the **static methods** below that can be used to retrieve their records. These methods are **faster** because they do not depend on SOQL and return data **directly** from the call.

STATIC METHODS	USE	EXAMPLE
<code>getAll()</code>	Used to retrieve a map of all custom metadata records of a specific custom metadata type.	<code>MyCoolAppSettings__mdt.getAll();</code>
<code>getInstance(recordId)</code>	Used to retrieve a single custom metadata type record by specifying its record ID .	<code>MyCoolAppSettings__mdt.getInstance('m0000000000123');</code>

Custom Metadata Type Methods

STATIC METHODS	USE	EXAMPLE
<code>getInstance(developerName)</code>	Used to retrieve a single custom metadata type record by specifying its developerName .	<code>MyCoolAppSettings__mdt.getInstance('Enable_Advanced_Search');</code>
<code>getInstance(qualifiedApiName)</code>	Used to retrieve a single custom metadata type record by specifying its qualified API name , which is a concatenation of the namespace prefix and developerName.	<code>MyCoolAppSettings__mdt.getInstance('FocusOnForce_Enable_Advanced_Search');</code>

Use Cases & Examples

Table of Contents



Use Cases of Custom Metadata Types

The following are some **use cases** for custom metadata types:



FREQUENCY

There is data that **does not change frequently** but needs to be **referenced frequently**.



SAME VALUE

The **value** of the setting is the same for **all users** and **profiles**.



PACKAGED

The values are tied to a **package** and are the **same** for any **subscribing organization**.

Use Cases of Custom Metadata Types



CUSTOM FIELD

A common **picklist** or **long text area** is needed for a package.



LINKED SETTINGS

Configuration data is **linked** by **relationships**.



BUSINESS RULES

Business rules must be **stored**.

Use Cases of Custom Metadata Types



DATA MAPPING

Mappings between **configuration data** need to be stored.



FORMULA PARAMETERS

Parameters for formulas used in **formula fields** can be stored in custom metadata types.



DEFAULT FIELD VALUES

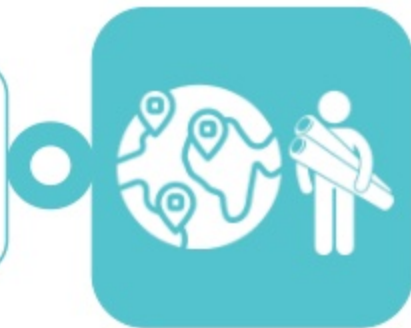
Default Field Values that are **uniform** to **multiple fields** can be set so that there will be no need to configure **each field** when a change of default value is required.

Examples of Custom Metadata Types

Here are some **example uses** for custom metadata types:

ISO CODE MIGRATION

A list of ISO country codes can be created using a custom metadata type. Then the list can be **migrated easily** to another environment, such as **Production**, using a **change set**.



BONUS/SALARY PLANS

A custom metadata type can be created for **employee bonus**, and it can be **related** to another **custom metadata type** for **salary plans** using a metadata relationship field.

Examples of Custom Metadata Types

PRODUCT CONSTANTS

A custom metadata type can be created to store information about **products** such as **machines**. Records of the metadata type can be used in **formula fields** to perform **calculations** based on the information.



Area Code to State Mapping

The example custom metadata type below can be used to identify a **state** by **area code**, or vice versa, and is accessible to all **Apex code** and **APIs**, as well as in **Setup**.

Singular Label	Area Code to State Mapping	Description
Plural Label	Area Code to State Mappings	Visibility Public
Object Name	Area_Code_to_State_Mapping	Protection Level
API Name	Area_Code_to_State_Mapping__mdt	Record Size 269
Created By	Martin Gessner, 9/17/2020 11:38 PM	Modified By Martin Gessner, 9/17/2020 11:38 PM

Standard Fields

Action	Field Label	Field Name	Data Type	Indexed
	Created By	CreatedBy	Lookup(User)	
Edit	Custom Metadata Record Name	DeveloperName	Text(40)	
Edit	Label	MasterLabel	Text(40)	
	Last Modified By	LastModifiedBy	Lookup(User)	
Edit	Namespace Prefix	NamespacePrefix	Text	
Edit	Protected Component	IsProtected	Checkbox	

Custom Fields

[New](#)

Action	Field Label	API Name	Data Type	Field Manageability	Indexed	Controlling Field	Modified By
Edit Del	State Code	State_Code__c	Text(128)	Upgradable			Martin Gessner, 9/17/2020 11:44 PM

Visibility Options

The different visibility options are **provided** when **configuring** the custom metadata type.

Custom Metadata Type Information

The singular and plural labels are used in tabs, page layouts, and reports.

Label Example: Example: Account Setting

Plural Label Example: Example: Account Settings

Starts with vowel sound ☒

The Object Name is used when referencing the object via the API.

Object Name Example: Account_Setting

Description

Visibility for custom metadata types can be set only in a developer, Sandbox, or scratch org. The visibility setting affects objects released in managed packages. Protected custom metadata types aren't accessible by a subscriber's org, but public custom metadata types are.

Visibility

- ☒ All Apex code and APIs can use the type, and it's visible in Setup.
- ☐ Only Apex code in the same namespace can see the type. The name of the type and the record are visible if they're referenced in a formula.
- ☐ Only Apex code in the same managed package can see the type. The name of the type and the record are visible if they're referenced in a formula.

Field Manageability Options

The different field manageability options are **provided** when configuring a **custom field** for the custom metadata type.

Field Information

Field Label	State Code	Data Type	Text
Field Name	State_Code		
Description			
Help Text			

General Options

Required

☐ Always require a value in this field in order to save a record

Unique

☐ Do not allow duplicate values

☒ Treat "ABC" and "abc" as duplicate values (case insensitive)

☐ Treat "ABC" and "abc" as different values (case sensitive)

Field Manageability

Who can change field values after records are installed via managed package?

☒ Only the package developer (via package upgrade)

☐ Any user with the Customize Application permission (managed package upgrades won't overwrite the value)

☐ No one

Default Value

[Show Formula Editor](#)

Manage Custom Metadata Type Records

A “Manage” **button** is provided on the configuration page of the custom metadata type to **create**, **update**, or **delete** records.

Custom Metadata Type

Area Code to State Mapping

Help for this Page ?

Standard Fields [6] | Custom Fields [1] | Validation Rules [0] | Page Layouts [1]

Custom Metadata Type Detail

EditDeleteManage Area Code to State Mappings

Singular Label	Area Code to State Mapping	Description	
Plural Label	Area Code to State Mappings	Visibility	Public
Object Name	Area_Code_to_State_Mapping	Protection Level	
API Name	Area_Code_to_State_Mapping__mdt	Record Size	269
Created By	Martin Gessner, 9/17/2020 11:38 PM	Modified By	Martin Gessner, 9/17/2020 11:38 PM

Standard Fields

Action	Field Label	Field Name	Data Type	Indexed
	<u>Created By</u>	CreatedBy	Lookup(User)	
Edit	<u>Custom Metadata Record Name</u>	DeveloperName	Text(40)	
Edit	<u>Label</u>	MasterLabel	Text(40)	
	<u>Last Modified By</u>	LastModifiedBy	Lookup(User)	
Edit	<u>Namespace Prefix</u>	NamespacePrefix	Text	
Edit	<u>Protected Component</u>	IsProtected	Checkbox	

Retrieve Custom Metadata Type Records Using SOQL

The code snippet below shows an example Apex code containing an **SOQL query** that is used to **retrieve** custom metadata type **records** of area-code-to-state mappings.

```
// retrieve a subset of metadata records
List<Area_Code_to_State_Mapping__mdt> vaCodes = [
    SELECT Label, State_Code__c
    FROM Area_Code_to_State_Mapping__mdt
    WHERE State_Code__c = 'VA'
];

// iterate through returned records
For (Area_Code_to_State_Mapping__mdt vaCode : vaCodes){
    // perform processing
}
```

Retrieve Custom Metadata Type Records Using Static Methods

This illustration demonstrates retrieving the same custom metadata type records using the static `getAll()` method. In this approach, no SOQL statement is involved.

```
List<Area_Code_to_State_Mapping__mdt> codes = Area_Code_to_State_Mapping__mdt.getAll().values();
List<Area_Code_to_State_Mapping__mdt> vaCodes = new List<Area_Code_to_State_Mapping__mdt>();
for (Area_Code_to_State_Mapping__mdt code : codes) {
    if (code.State_Code__c == 'VA') { vaCodes.add(code); }
}

// iterate through list collection
For (Area_Code_to_State_Mapping__mdt vaCode : vaCodes){
    // perform processing
}
```

Storing Product Specifications

In this example, the custom metadata type is used to store product specifications related to **machines**.

The screenshot displays the Salesforce Setup interface for managing Custom Metadata Types. The left sidebar shows the navigation menu with 'Setup', 'Home', and 'Object Manager'. The 'Custom Metadata Types' link is highlighted. The main content area shows the 'Machine' custom metadata type. A red box highlights the 'Machine' title. A red box highlights the 'Custom Metadata Types' link in the sidebar. A red box highlights the 'Machine Edit' form. A red box highlights the 'Records of a custom metadata type can be created to store metadata information in an org.' message. The 'Machine Edit' form includes fields for Label, Machine Name, Weight (lbs), Positioning Speed (m/min), Operating Range (mm), Marking Speed (m/min), Working Area (sq. mm), and Type. The 'Machine Name' field is highlighted with a red box. The 'Type' field is set to 'Beam Processing'. The 'Protected Component' checkbox is checked. The 'Namespace Prefix' field is empty. The 'Machine Edit' form also includes 'Save', 'Save & New', and 'Cancel' buttons. A red box highlights the 'Machine Edit' title. A red box highlights the 'Machine Name' field. A red box highlights the 'Type' field. A red box highlights the 'Beam Processing' value in the 'Type' field. A red box highlights the 'Machine Edit' form. A red box highlights the 'Records of a custom metadata type can be created to store metadata information in an org.' message.

Setup Home Object Manager

Custom Metadata Types

Custom Code

Custom Metadata Types

Didn't find what you're looking for?
Try using Global Search.

Machine

Help for this Page

Machine Edit

Save Save & New Cancel

Information

Label Rex Beam V100

Machine Name Rex_Beam_V100

Weight (lbs) 200

Positioning Speed (m/min) 10

Operating Range (mm) 500

Marking Speed (m/min) 50

Working Area (sq. mm) 100

Type Beam Processing

Protected Component ☒

Namespace Prefix

= Required Information

Referencing the Custom Metadata Type Record

A custom metadata can be referenced in a **formula** by selecting the **\$CustomMetadata** option when inserting a field.

Insert Field

Select a field, then click Insert. Labels followed by a ">" indicate that there are more fields available.

Account >
\$Api >
\$CustomMetadata >
\$Organization >
\$Profile >
\$System >
\$User >
\$UserRole >

Machine >

Rex Beam V100 >

Custom Metadata Record ID
Custom Metadata Record Name
Label
Marking Speed (m/min)
Namespace Prefix
Operating Range (mm)
Positioning Speed (m/min)
Qualified API Name
Type

\$CustomMetadata.MachIr

API Name: \$CustomMetadata

A custom metadata type record can be referenced in a formula field.

Using the Custom Metadata Type Record in Formula Field

In the example below, a **mathematical operation** is performed using **two field values** of a custom metadata record.

Field LabelMachine's Working Range

Field NameMachine_s_Working_Ra

Description

Help Text

Formula Options

Formula Return TypeNumber

Decimal Places0

Enter your formula and click Check Syntax to check for errors. Click the Advanced Formula subtab to use additional fields, operators, and functions.

Example: Fahrenheit = 1.8 * Celsius_c + 32 More Examples ...

Simple FormulaAdvanced Formula

Insert FieldInsert Operator

Machine's Working Range (Number) =
\$CustomMetadata.Machine__mdt.Rex_Beam_V100.Operating_Range_mm__c *
\$CustomMetadata.Machine__mdt.Rex_Beam_V100.Working_Area__c

Quick Tips
Operators & Functions

Parameters stored in a custom metadata type record can be used in a formula field.

ABS
ADDMONTHS
AND

Using Custom Metadata Type Record in Field Default Values

Custom metadata types are **supported** in field default values, although the **\$CustomMetadata** option is **not available** in the field type dropdown.

General Options

Required

Unique

External ID

Default Value

The formula editor does not include custom metadata field types and has to be defined in the Formula Editor manually.

Hide Formula Editor

Select Field Type

-- Select field type --

Insert Field

-- Insert Merge Field --

Insert Operator

\$CustomMetadata.Field_Default_Value_mdt.Discount.Default_Value_c

Check Syntax

No syntax errors in merge fields or functions.

Custom Metadata Type API Name

Record API Name

Field API Name

Select Field Type

✓ -- Select field type --

\$Label

\$Organization

\$Permission

\$Profile

\$Setup.ReviewFrequency__c

\$Setup.Run_Validation_Rules__c

\$System

\$User

\$UserRole

BLANKVALUE

BR

Insert Selected Function

Custom Settings vs. Custom Metadata Types

[Table of Contents](#)



Differences Between Custom Settings and Custom Metadata Types

The following tables highlights the **differences** between custom settings and custom metadata types:

FEATURE	CUSTOM SETTINGS	CUSTOM METADATA TYPES
Configuration deployable	✓	✓
Configurable data deployable	✗	✓
Visible in test classes	✗	✓
Create/update in bulk custom metadata loader	✗	✓
Supports relationships	✗	✓
Can use picklist as a field type (except global)	✗	✓

Differences Between Custom Settings and Custom Metadata Types

FEATURE	CUSTOM SETTINGS	CUSTOM METADATA TYPES
Can use long text area as a field type	✗	✓
Can use currency as a field type	✓	✗
Exempt from SOQL limits	✓	✓
Supports hierarchy configuration	✓	✗
Create, read, update and delete values via Apex	✓	✓
Accessible by formula fields (hierarchy)	✓	✓

Protecting Custom Settings & Custom Metadata Types

To protect the **data** in the org, **Schema Settings** should be configured so that custom settings & custom metadata types can be accessed, for example, only in **Apex**, **flows**, and **formulas**.

 **Schema Settings**

Restrict access to custom metadata types

Allow access only through granted permissions and when accessing custom metadata types in system mode, for example, from Apex.



Restrict access to custom settings

Allow access only through granted permissions and when accessing custom settings in system mode, for example, from Apex.



Enable SOSL on custom settings

Enable SOSL searches to return data from custom settings.

Manage list custom settings type ⓘ

Enable list as a setting type option on the New Custom Setting Definition page.

When the settings are activated, users will be required to have the **Customize Application** permission in order to access the data. Also, SOAP API will be prevented access.

Learn More

-  [Custom Metadata Types](#)
-  [Custom Metadata Types \(CustomObject\)](#)
-  [Custom Settings](#)
-  [Custom Settings Methods](#)
-  [Create Custom Data Sets](#)
-  [Access Custom Settings](#)



Learn More

-  [Use Custom Metadata Types to Save Years of Development](#)
-  [Custom Metadata Types Implementation Guide](#)
-  [Testing Custom Metadata Types](#)
-  [Platform on the Platform](#)
-  [Custom Metadata Types and Advanced Formula Fields](#)

